SZCZUREK,SANTINI,FIACCHI - HM 5

Aleksandra Szczurek, Lucia Santini, Giada Fiacchi

2025-11-19

EXERCISE 1

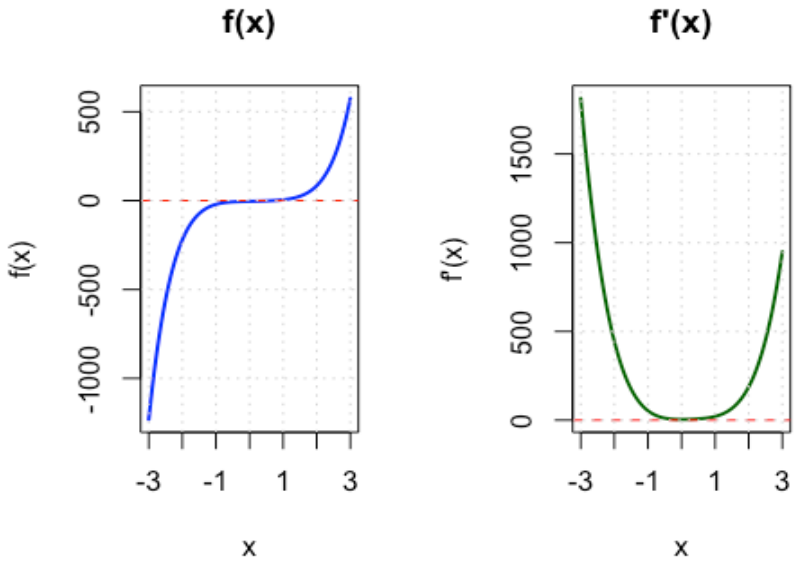
Write an R code to find the root of f(x) = 3x^5 − 4x^4 + 6x^3 + 4x − 4.

a) Write functions for f(x) and for the corresponding derivative f'(x)

```
f <- function(x) 3*x^5 - 4*x^4 + 6*x^3 + 4*x - 4
fprime <- function(x) 15*x^4 - 16*x^3 + 18*x^2 + 4

# Visualization of functions
par(mfrow = c(1, 2))
curve(f, from = -3, to = 3, main = "f(x)",
      ylab = "f(x)", xlab = "x", col = "blue", lwd = 2)
abline(h = 0, col = "red", lty = 2)
grid()

curve(fprime, from = -3, to = 3, main = "f'(x)",
      ylab = "f'(x)", xlab = "x", col = "darkgreen", lwd = 2)
abline(h = 0, col = "red", lty = 2)
grid()
```



b) Write a function that carries out the Newton Raphson method and takes as input both functions of point (a)

c) Use loops inside the Newton-Raphson function to check convergence

d) The code should be flexible so that to find the root of any function in input

```r
# Newton-Raphson
newton_raphson <- function(f, fprime, x0, tol = 1e-10, nmax = 100) {

  xn <- x0
  out <- data.frame(
    iter = 0,
    x = xn,
    fx = f(xn),
    dfx = fprime(xn),
    x_next = NA
  )
  i <- 0
  while (i < nmax) {

    fx  <- f(xn)
    dfx <- fprime(xn)

    if (dfx == 0) stop("Null derivative - impossible method")

    xn1 <- xn - fx/dfx

    out <- rbind(out, data.frame(
      iter = i,
```

```
      x = xn,
      fx = fx,
      dfx = dfx,
      x_next = xn1
    ))

    if (abs(f(xn1)) < tol) break

    xn <- xn1
    i <- i+1
  }

  return(out)
}

# Compute the iterations starting from x0 = 1.4
tab <- newton_raphson(f, fprime, x0 = 1.4)
print(tab)
```

```
##   iter         x           fx      dfx    x_next
## 1    0 1.4000000 1.883232e+01 53.00000        NA
## 2    0 1.4000000 1.883232e+01 53.00000 1.0446732
## 3    1 1.0446732 5.987861e+00 23.26804 0.7873304
## 4    2 0.7873304 1.448242e+00 13.11302 0.6768874
## 5    3 0.6768874 1.549385e-01 10.43392 0.6620379
## 6    4 0.6620379 2.284617e-03 10.12814 0.6618123
## 7    5 0.6618123 5.139477e-07 10.12359 0.6618123
```

```
# Plot of the function with points and tangents
par(mfrow=c(1,1))
curve(f, from = -1.5, to = +2, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson (p= 1.4)")
abline(h = 0)

# extract x_n from the dataframe
xs <- tab$x
ys <- f(xs)

#points
points(xs, ys, pch = 19, col = "red")

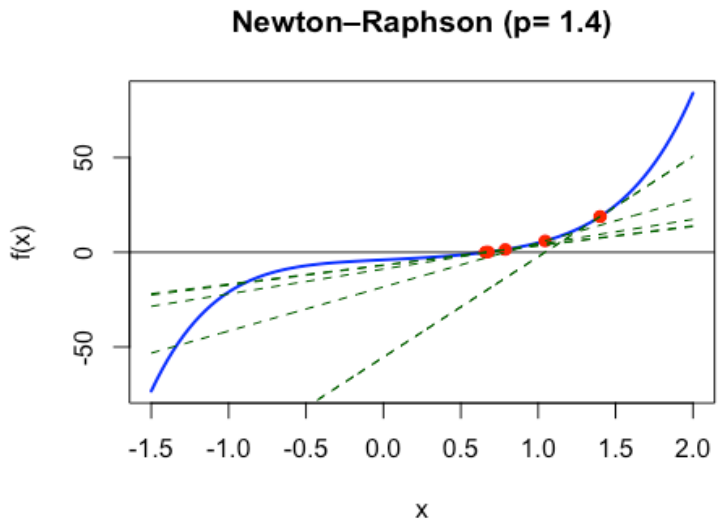
#tangents
for (i in 1:(nrow(tab)-1)) {

  x0 <- tab$x[i]
  y0 <- tab$fx[i]
```

```
  slope <- tab$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)
}
```



e) Plot the graph of the function and try different starting points for the NR #algorithm

```
#p = 1
#Compute the iterations starting from x0 = 1
tab1 <- newton_raphson(f, fprime, x0 = 1)
print(tab1)

##   iter         x           fx      dfx    x_next
## 1    0 1.0000000 5.000000e+00 21.00000        NA
## 2    0 1.0000000 5.000000e+00 21.00000 0.7619048
## 3    1 0.7619048 1.123650e+00 12.42711 0.6714855
## 4    2 0.6714855 9.888006e-02 10.32135 0.6619053
## 5    3 0.6619053 9.420926e-04 10.12547 0.6618123
## 6    4 0.6618123 8.742017e-08 10.12359 0.6618123

# Plot of the function with points and tangents
curve(f, from = -1.5, to = +2, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson (p= 1)")
abline(h = 0)

# extract x_n from the dataframe
xs <- tab1$x
ys <- f(xs)
```

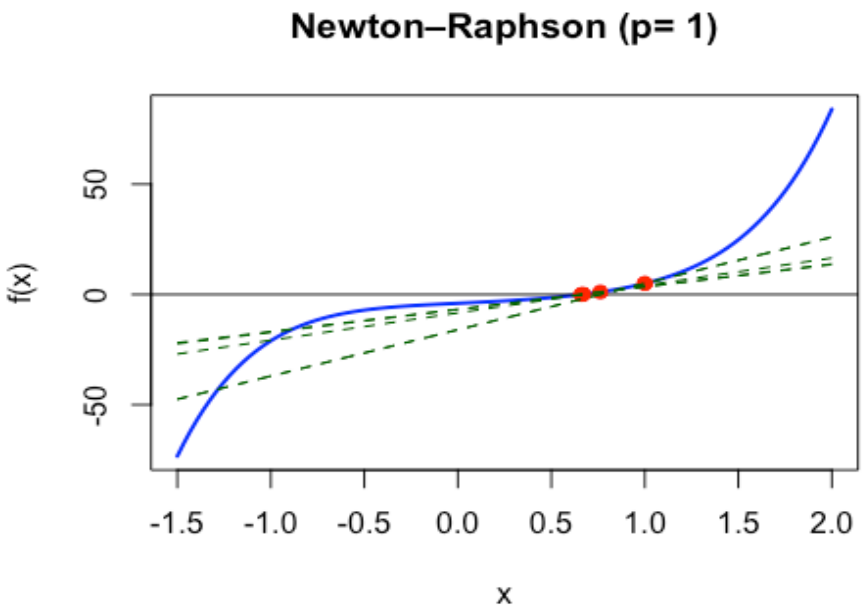
```r
# points
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(tab1)-1)) {

  x0 <- tab1$x[i]
  y0 <- tab1$fx[i]
  slope <- tab1$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)
}
```




```r
#p= 0.5
# Compute the iterations starting from x0 = 0.5
tab2 <- newton_raphson(f, fprime, x0 = 0.5)
print(tab2)
```

```
##   iter         x            fx      dfx    x_next
## 1    0 0.5000000 -1.406250e+00  7.43750        NA
## 2    0 0.5000000 -1.406250e+00  7.43750 0.6890756
## 3    1 0.6890756  2.836844e-01 10.69369 0.6625474
## 4    2 0.6625474  7.447805e-03 10.13845 0.6618128
## 5    3 0.6618128  5.455580e-06 10.12360 0.6618123
```

```r
# Plot of the function with points and tangents
curve(f, from = -1.5, to = +2, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson (p= 0.5)")
abline(h = 0)

# extract x_n from the dataframe
xs <- tab2$x
ys <- f(xs)

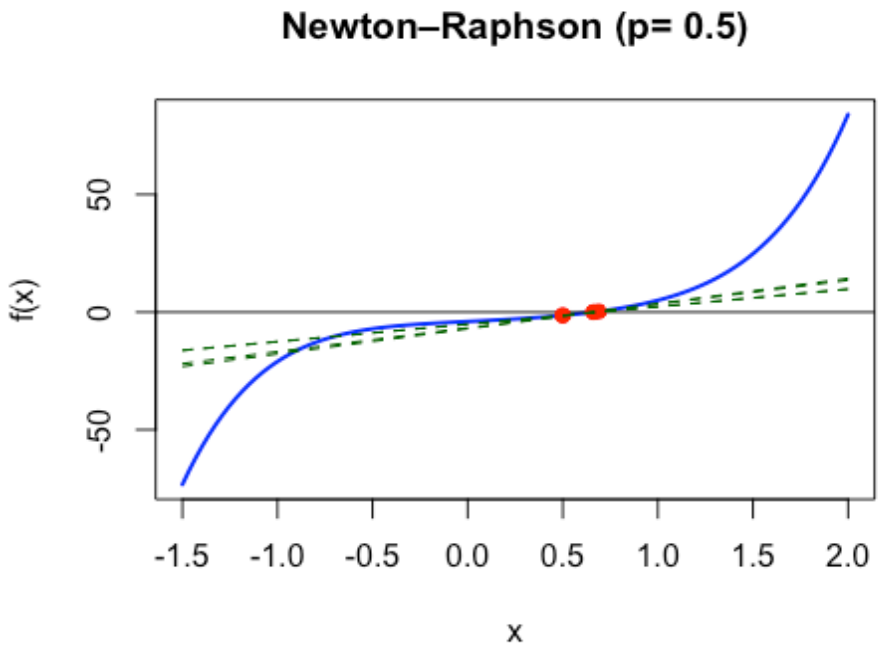
# points
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(tab2)-1)) {

  x0 <- tab2$x[i]
  y0 <- tab2$fx[i]
  slope <- tab2$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)
}
```




```r
#p= -0.5
# Compute the iterations starting from x0 = -0.5
```

```r
tab3 <- newton_raphson(f, fprime, x0 = -0.5)
print(tab3)
```

```
##   iter          x            fx       dfx    x_next
## 1    0 -0.5000000 -7.093750e+00 11.437500        NA
## 2    0 -0.5000000 -7.093750e+00 11.437500 0.1202186
## 3    1  0.1202186 -3.509461e+00  4.235479 0.9488052
## 4    2  0.9488052  3.985202e+00 18.694103 0.7356256
## 5    3  0.7356256  8.058883e-01 11.763897 0.6671203
## 6    4  0.6671203  5.402245e-02 10.231503 0.6618403
## 7    5  0.6618403  2.840368e-04 10.124154 0.6618123
## 8    6  0.6618123  7.947648e-09 10.123587 0.6618123
```

```r
# Plot of the function with points and tangents
curve(f, from = -1.5, to = +2, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson (p= -0.5)")
abline(h = 0)

# extract x_n from the dataframe
xs <- tab3$x
ys <- f(xs)

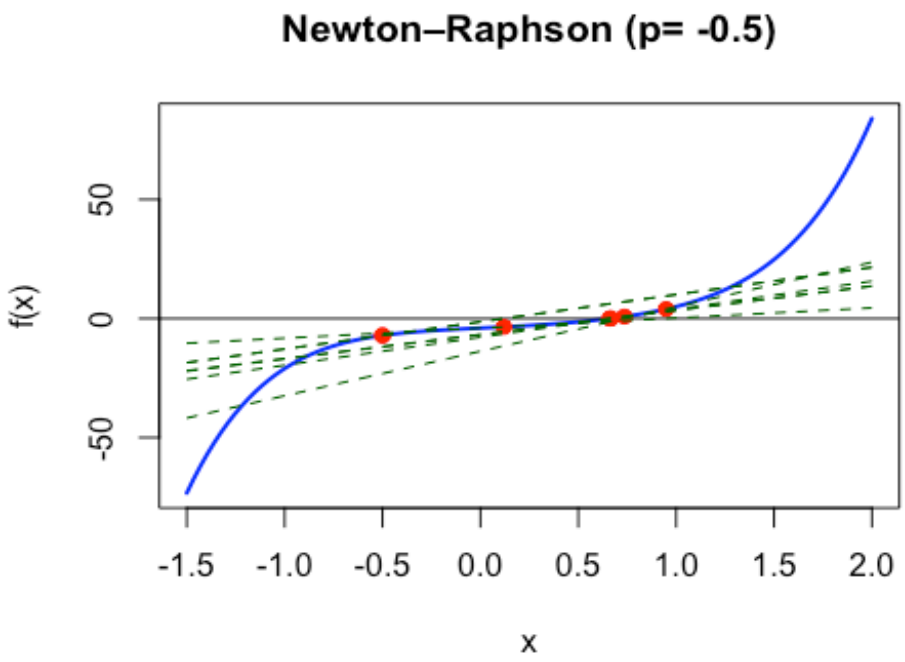
# points
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(tab3)-1)) {

  x0 <- tab3$x[i]
  y0 <- tab3$fx[i]
  slope <- tab3$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)
}
```




```
#p= -1
# Compute the iterations starting from x0 = -1
tab4 <- newton_raphson(f, fprime, x0 = -1)
print(tab4)
```

```
##   iter          x            fx       dfx     x_next
## 1    0 -1.0000000 -2.100000e+01 53.000000         NA
## 2    0 -1.0000000 -2.100000e+01 53.000000 -0.6037736
## 3    1 -0.6037736 -8.507975e+00 16.076753 -0.0745638
## 4    2 -0.0745638 -4.300873e+00  4.107172  0.9725978
## 5    3  0.9725978  4.442170e+00 19.728850  0.7474367
## 6    4  0.7474367  9.465521e-01 12.056424  0.6689265
## 7    5  0.6689265  7.253574e-02 10.268557  0.6618627
## 8    6  0.6618627  5.099530e-04 10.124604  0.6618123
## 9    7  0.6618123  2.561694e-08 10.123587  0.6618123
```

```
# Plot of the function with points and tangents
curve(f, from = -1.5, to = +2, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson (p= -1)", ylim=c(-50, +50))
abline(h = 0)

# extract x_n from the dataframe
xs <- tab4$x
ys <- f(xs)

# points
```

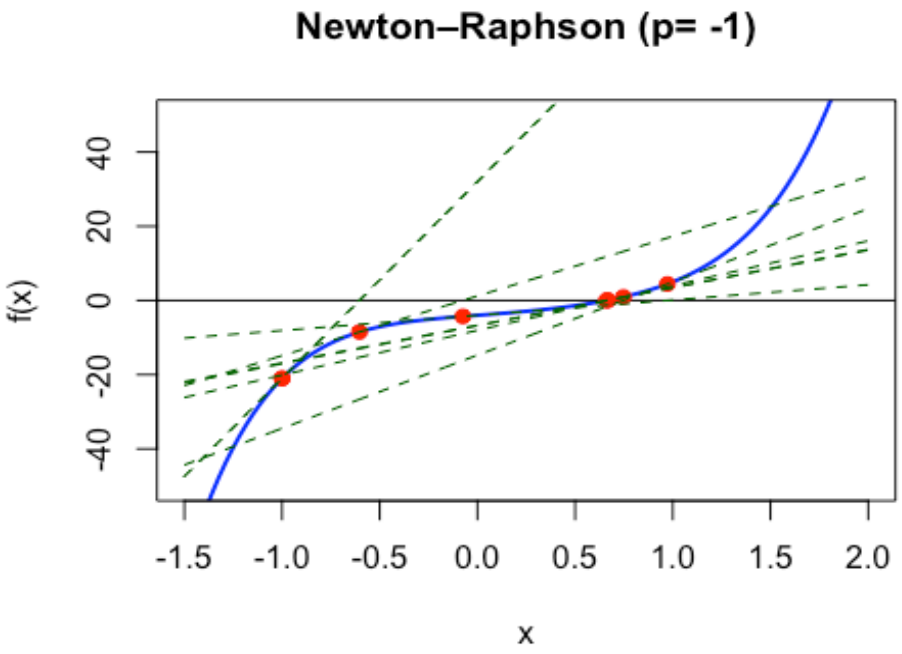
```r
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(tab4)-1)) {

  x0 <- tab4$x[i]
  y0 <- tab4$fx[i]
  slope <- tab4$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)
}
```



EXERCISE 2

Use the code implemented in the previous point to find the root of the following function:

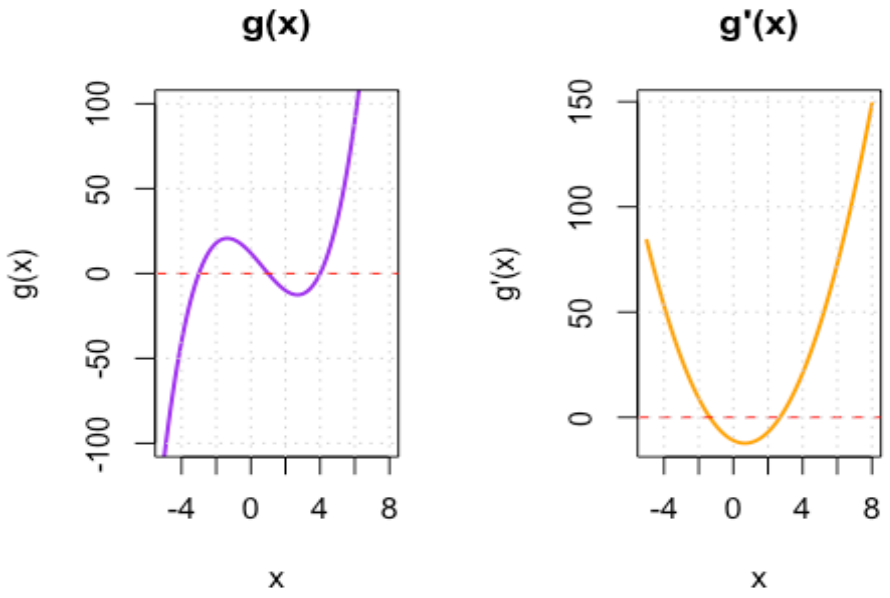
$f(x) = x^{3-2x}2-11x+12$,

with starting values x0 = 2.35287527 and x0 = 2.35284172. See what happens with the suggested choice of starting values and try also additional ones based on the graph of the corresponding function.

```r
# Write functions for g(x) and for the corresponding derivative g'(x)
g <- function(x) x^3 - 2*x^2 - 11*x + 12
gprime <- function(x) 3*x^2 - 4*x - 11  #first derivative

# Visualization of the functions
par(mfrow=c(1,2))
curve(g, from = -5, to = 8, main = "g(x)", ylim=c(-100,100),
      ylab = "g(x)", xlab = "x", col = "purple", lwd = 2)
abline(h = 0, col = "red", lty = 2)
grid()

curve(gprime, from = -5, to = 8, main = "g'(x)",
      ylab = "g'(x)", xlab = "x", col = "orange", lwd = 2)
abline(h = 0, col = "red", lty = 2)
grid()
```




```r
# Newton-Raphson
newton_raphson <- function(f, fprime, x0, tol = 1e-10, nmax = 100) {

  xn <- x0
```

```r
  out <- data.frame(
    iter = 0,
    x = xn,
    fx = f(xn),
    dfx = fprime(xn),
    x_next = NA
  )
  i <- 0
  while (i < nmax) {

    fx  <- f(xn)
    dfx <- fprime(xn)

    if (dfx == 0) stop("Null derivative - impossible method")

    xn1 <- xn - fx/dfx

    out <- rbind(out, data.frame(
      iter = i,
      x = xn,
      fx = fx,
      dfx = dfx,
      x_next = xn1
    ))

    if (abs(f(xn1)) < tol) break

    xn <- xn1
    i <- i+1
  }

  return(out)
}

# Compute the iterations starting from x0 = 2.35287527
table <- newton_raphson(g, gprime, x0 = 2.35287527)
print(table)
```

```
##    iter             x            fx           dfx        x_next
## 1     0  2.352875e+00 -1.192810e+01 -3.803435e+00           NA
## 2     0  2.352875e+00 -1.192810e+01 -3.803435e+00 -7.832642e-01
## 3     1 -7.832642e-01  1.890837e+01 -6.026434e+00  2.354307e+00
## 4     2  2.354307e+00 -1.193354e+01 -3.788946e+00 -7.952599e-01
## 5     3 -7.952599e-01  1.898003e+01 -5.921646e+00  2.409935e+00
## 6     4  2.409935e+00 -1.212847e+01 -3.216379e+00 -1.360911e+00
```

```
## 7     5 -1.360911e+00  2.074535e+01 -1.246248e-04  1.664611e+05
## 8     6  1.664611e+05  4.612468e+15  8.312727e+10  1.109743e+05
## 9     7  1.109743e+05  1.366657e+15  3.694545e+10  7.398310e+04
## 10    8  7.398310e+04  4.049355e+14  1.642020e+10  4.932229e+04
## 11    9  4.932229e+04  1.199809e+14  7.297867e+09  3.288175e+04
## 12   10  3.288175e+04  3.554989e+13  3.243496e+09  2.192139e+04
## 13   11  2.192139e+04  1.053330e+13  1.441554e+09  1.461448e+04
## 14   12  1.461448e+04  3.120978e+12  6.406907e+08  9.743210e+03
## 15   13  9.743210e+03  9.247342e+11  2.847514e+08  6.495696e+03
## 16   14  6.495696e+03  2.739953e+11  1.265562e+08  4.330686e+03
## 17   15  4.330686e+03  8.118379e+10  5.624720e+07  2.887347e+03
## 18   16  2.887347e+03  2.405445e+10  2.499876e+07  1.925121e+03
## 19   17  1.925121e+03  7.127242e+09  1.111056e+07  1.283638e+03
## 20   18  1.283638e+03  2.111774e+09  4.938032e+06  8.559829e+02
## 21   19  8.559829e+02  6.257096e+08  2.194685e+06  5.708807e+02
## 22   20  5.708807e+02  1.853947e+08  9.754197e+05  3.808141e+02
## 23   21  3.808141e+02  5.493123e+07  4.335240e+05  2.541055e+02
## 24   22  2.541055e+02  1.627557e+07  1.926814e+05  1.696367e+02
## 25   23  1.696367e+02  4.822162e+06  8.564028e+04  1.133295e+02
## 26   24  1.133295e+02  1.428635e+06  3.806643e+04  7.579947e+01
## 27   25  7.579947e+01  4.231974e+05  1.692248e+04  5.079147e+01
## 28   26  5.079147e+01  1.253242e+05  7.525153e+03  3.413743e+01
## 29   27  3.413743e+01  3.708828e+04  3.348542e+03  2.306148e+01
## 30   28  2.306148e+01  1.095948e+04  1.492249e+03  1.571720e+01
## 31   29  1.571720e+01  3.227678e+03  6.672227e+02  1.087972e+01
## 32   30  1.087972e+01  9.434007e+02  3.005861e+02  7.741183e+00
## 33   31  7.741183e+00  2.708926e+02  1.378130e+02  5.775530e+00
## 34   32  5.775530e+00  7.440854e+01  6.596811e+01  4.647582e+00
## 35   33  4.647582e+00  1.806443e+01  3.520974e+01  4.134530e+00
## 36   34  4.134530e+00  3.008553e+00  2.374490e+01  4.007827e+00
## 37   35  4.007827e+00  1.649819e-01  2.115673e+01  4.000029e+00
## 38   36  4.000029e+00  6.090547e-04  2.100058e+01  4.000000e+00
## 39   37  4.000000e+00  8.411099e-09  2.100000e+01  4.000000e+00
```

```r
# Plot of the function with points and tangents
par(mfrow=c(1,1))
curve(g, from = -8, to = +10, col = "purple", lwd = 2,
      ylab = "g(x)", main = "Newton-Raphson (x0 = 2.35287527)")
abline(h = 0)

# extract x_n from the dataframe
xs <- table$x
ys <- g(xs)

# points
```

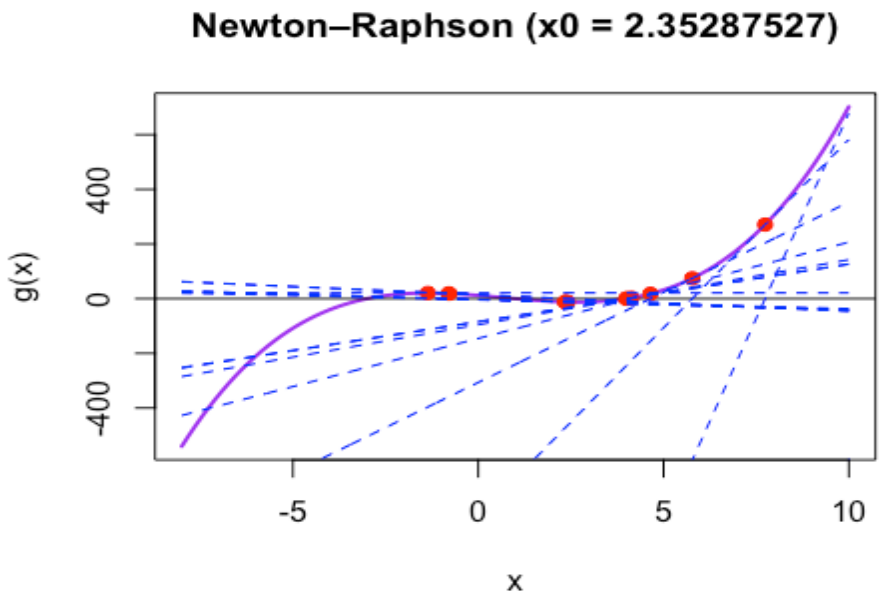
```r
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(table)-1)) {

  x0 <- table$x[i]
  y0 <- table$fx[i]
  slope <- table$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "blue", lty = 2, n = 1000)
}
```




```r
# Compute the iterations starting from x0 = 2.35284172
table2 <- newton_raphson(g, gprime, x0 = 2.35284172)
print(table2)
```

```
##    iter             x            fx           dfx        x_next
## 1     0  2.352842e+00 -1.192798e+01 -3.803774e+00            NA
## 2     0  2.352842e+00 -1.192798e+01 -3.803774e+00 -7.829844e-01
## 3     1 -7.829844e-01  1.890668e+01 -6.028869e+00  2.353040e+00
## 4     2  2.353040e+00 -1.192873e+01 -3.801769e+00 -7.846387e-01
## 5     3 -7.846387e-01  1.891664e+01 -6.014471e+00  2.360549e+00
## 6     4  2.360549e+00 -1.195699e+01 -3.725621e+00 -8.488466e-01
## 7     5 -8.488466e-01  1.928460e+01 -5.442992e+00  2.694168e+00
## 8     6  2.694168e+00 -1.259720e+01 -1.044714e-03 -1.205535e+04
```

```
## 9     7 -1.205535e+04 -1.752312e+12  4.360425e+08 -8.036677e+03
## 10    8 -8.036677e+03 -5.192035e+11  1.937967e+08 -5.357563e+03
## 11    9 -5.357563e+03 -1.538381e+11  8.613186e+07 -3.571487e+03
## 12   10 -3.571487e+03 -4.558164e+10  3.828083e+07 -2.380770e+03
## 13   11 -2.380770e+03 -1.350567e+10  1.701371e+07 -1.586959e+03
## 14   12 -1.586959e+03 -4.001677e+09  7.561652e+06 -1.057752e+03
## 15   13 -1.057752e+03 -1.185681e+09  3.360738e+06 -7.049484e+02
## 16   14 -7.049484e+02 -3.513118e+08  1.493666e+06 -4.697473e+02
## 17   15 -4.697473e+02 -1.040918e+08  6.638555e+05 -3.129485e+02
## 18   16 -3.129485e+02 -3.084157e+07  2.950510e+05 -2.084188e+02
## 19   17 -2.084188e+02 -9.137955e+06  1.311379e+05 -1.387368e+02
## 20   18 -1.387368e+02 -2.707350e+06  5.828765e+04 -9.228872e+01
## 21   19 -9.228872e+01 -8.020494e+05  2.590978e+04 -6.133325e+01
## 22   20 -6.133325e+01 -2.375583e+05  1.151963e+04 -4.071121e+01
## 23   21 -4.071121e+01 -7.032987e+04  5.124054e+03 -2.698578e+01
## 24   22 -2.698578e+01 -2.079954e+04  2.281640e+03 -1.786973e+01
## 25   23 -1.786973e+01 -6.136382e+03  1.018461e+03 -1.184458e+01
## 26   24 -1.184458e+01 -1.800022e+03  4.572606e+02 -7.908045e+00
## 27   25 -7.908045e+00 -5.206327e+02  2.082437e+02 -5.407933e+00
## 28   26 -5.407933e+00 -1.451632e+02  9.836895e+01 -3.932231e+00
## 29   27 -3.932231e+00 -3.647225e+01  5.111626e+01 -3.218716e+00
## 30   28 -3.218716e+00 -6.660706e+00  3.295526e+01 -3.016602e+00
## 31   29 -3.016602e+00 -4.678963e-01  2.836607e+01 -3.000107e+00
## 32   30 -3.000107e+00 -3.001972e-03  2.800236e+01 -3.000000e+00
## 33   31 -3.000000e+00 -1.264227e-07  2.800000e+01 -3.000000e+00
```

```r
# Plot of the function with points and tangents
curve(g, from = -10, to = +8, col = "purple", lwd = 2,
      ylab = "g(x)", main = "Newton-Raphson (x0 = 2.35284172)")
abline(h = 0)

# extract x_n from the dataframe
xs <- table2$x
ys <- g(xs)

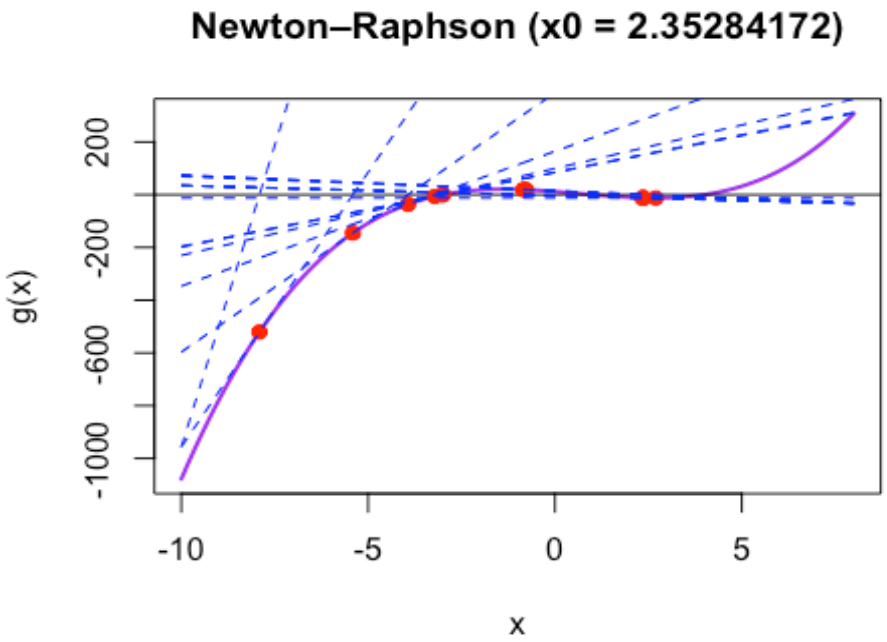
# points
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(table2)-1)) {

  x0 <- table2$x[i]
  y0 <- table2$fx[i]
  slope <- table2$dfx[i]
```

```
  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "blue", lty = 2, n = 1000)
}
```



The two starting points lie on opposite sides of an unstable region of the Newton–Raphson method. Even though they differ only by a very small amount, one starts inside the region that converges to the root at x=4, while the other starts inside the region from which the method diverges.The function has three real roots, as seen in the plot: one around x≈−3, one around x≈1, one around x≈4.

Therefore, Newton–Raphson can converge to any of these three roots depending on the chosen starting point.Thus we also test:

x0 = –3.5 x0 = 1.5 x0 = 4

```
#p=-3.5
#Compute the iterations starting from x0 = -3.5
table3 <- newton_raphson(g, gprime, x0 = -3.5)
print(table3)
```

```
##   iter         x            fx      dfx    x_next
## 1    0 -3.500000 -1.687500e+01 39.75000        NA
## 2    0 -3.500000 -1.687500e+01 39.75000 -3.075472
## 3    1 -3.075472 -2.176293e+00 29.67747 -3.002140
## 4    2 -3.002140 -5.997581e-02 28.04710 -3.000002
## 5    3 -3.000002 -5.031971e-05 28.00004 -3.000000
```

```r
# Plot of the function with points and tangents
curve(g, from = -5, to = +5, col = "purple", lwd = 2,
      ylab = "g(x)", main = "Newton-Raphson (x0 = -3.5)")
abline(h = 0)

# extract x_n from the dataframe
xs <- table3$x
ys <- g(xs)

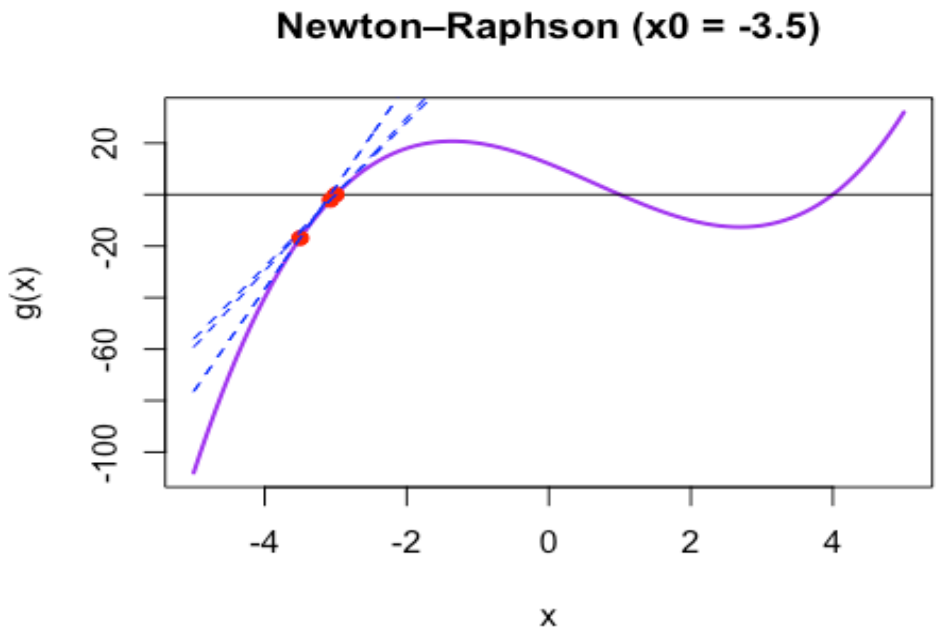
# points
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(table3)-1)) {

  x0 <- table3$x[i]
  y0 <- table3$fx[i]
  slope <- table3$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "blue", lty = 2, n = 1000)
}
```




```r
#p=1.5
# Compute the iterations starting from x0 = 1.5
```

```r
table4 <- newton_raphson(f, fprime, x0 = 1.5)
print(table4)
```

```
##   iter         x           fx      dfx    x_next
## 1    0 1.5000000 2.478125e+01 66.43750        NA
## 2    0 1.5000000 2.478125e+01 66.43750 1.1269991
## 3    1 1.1269991 8.097998e+00 28.15771 0.8394047
## 4    2 0.8394047 2.170638e+00 14.66661 0.6914061
## 5    3 0.6914061 3.086648e-01 10.74429 0.6626779
## 6    4 0.6626779 8.770242e-03 10.14109 0.6618130
## 7    5 0.6618130 7.562710e-06 10.12360 0.6618123
```

```r
# Plot of the function with points and tangents
curve(f, from = -1.5, to = +2, col = "purple", lwd = 2,
      ylab = "g(x)", main = "Newton-Raphson (x0 = 1.5)")
abline(h = 0)

# extract x_n from the dataframe
xs <- table4$x
ys <- f(xs)

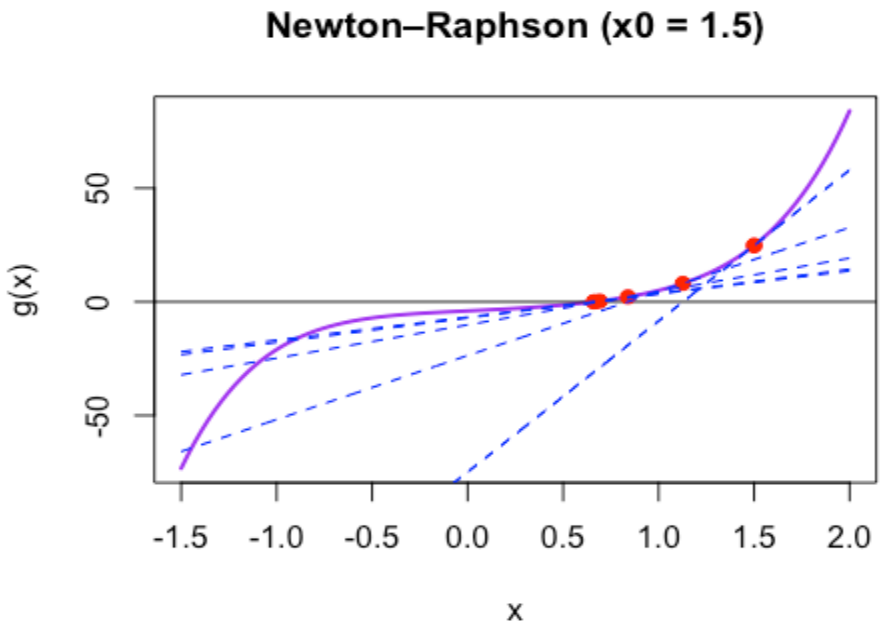
# points
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(tab)-1)) {

  x0 <- table4$x[i]
  y0 <- table4$fx[i]
  slope <- table4$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "blue", lty = 2, n = 1000)
}
```




```
#p=4
# Compute the iterations starting from x0 = 4
table5 <- newton_raphson(f, fprime, x0 = 4)
print(table5)
```

```
##    iter         x           fx        dfx    x_next
## 1     0 4.0000000 2.444000e+03 3108.00000        NA
## 2     0 4.0000000 2.444000e+03 3108.00000 3.2136422
## 3     1 3.2136422 8.096332e+02 1258.73058 2.5704282
## 4     2 2.5704282 2.701918e+02  506.00533 2.0364579
## 5     3 2.0364579 9.109794e+01  201.50474 1.5843696
## 6     4 1.5843696 3.094576e+01   80.06864 1.1978792
## 7     5 1.1978792 1.026795e+01   33.21149 0.8887106
## 8     6 0.8887106 2.934239e+00   16.34288 0.7091682
## 9     7 0.7091682 5.029938e-01   11.14000 0.6640162
## 10    8 0.6640162 2.236060e-02   10.16822 0.6618171
## 11    9 0.6618171 4.901027e-05   10.12368 0.6618123
## 12   10 0.6618123 2.366392e-10   10.12359 0.6618123
```

```
# Plot of the function with points and tangents
curve(f, from = -6, to = +6, col = "purple", lwd = 2,
      ylab = "g(x)", main = "Newton-Raphson (x0 = 4)")
abline(h = 0)

# extract x_n from the dataframe
xs <- table5$x
```

```r
ys <- f(xs)

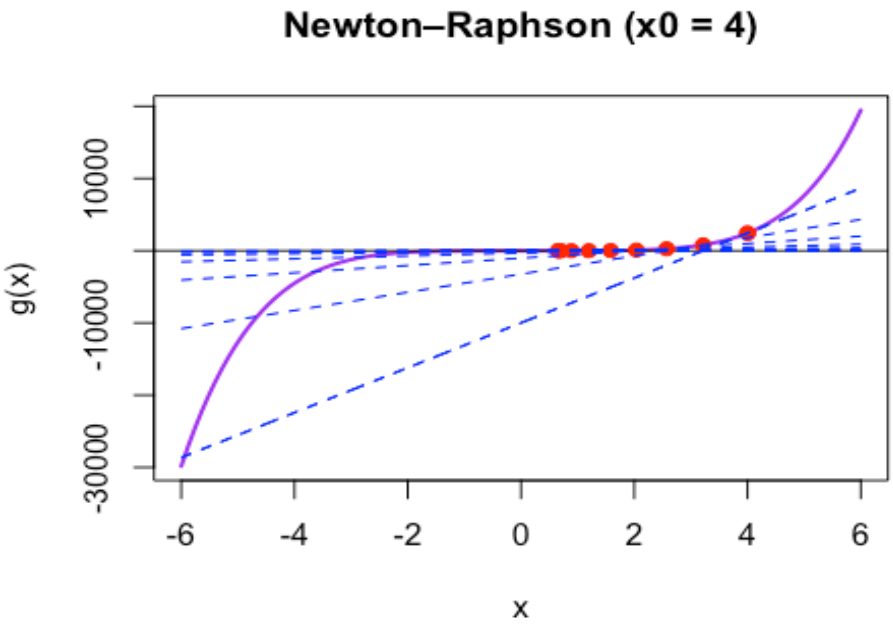
# points
points(xs, ys, pch = 19, col = "red")

# tangents
for (i in 1:(nrow(table5)-1)) {

  x0 <- table5$x[i]
  y0 <- table5$fx[i]
  slope <- table5$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "blue", lty = 2, n = 1000)
}
```



EXERCISE 3

Use the golden section search algorithm to maximize the following function:

f(x) = 4 sin(x)[1 + cos(x)]

starting with xL = 0 and xU = π/2. By analytical computation (Hint: you can use Excel) show all the updated values of x1, x2, xL, and xU for each iteration of the algorithm, up to convergence, setting the tolerance level at ϵ= 10−5.

Golden ratio (Phi) (SQRT(5)-1)/2	*Tolerance*	*Iteration*	*xl*	*d (Phi*(xu-xl))*	*x1 (xu-d)*	*x2 (xl+d)*	*xu*	*f(x1)*	*f(x2)*	*xu-xl (to check the tolerance)*
0.618033989	1.00E-04	1	0	0.970805519	0.599990807	0.970805519	1.570796327	4.122604393	5.165428069	1.570796327
		2	0.599990807	0.599990807	0.970805519	1.199981615	1.570796327	5.165428069	5.079110284	0.970805519
		3	0.599990807	0.370814712	0.829166903	0.970805519	1.199981615	4.941817594	5.165428069	0.599990807
		4	0.829166903	0.229176096	0.970805519	1.058342998	1.199981615	5.165428069	5.195508356	0.370814712
		5	0.970805519	0.141638616	1.058342998	1.112444136	1.199981615	5.195508356	5.174332792	0.229176096
		6	0.970805519	0.087537479	1.024906657	1.058342998	1.112444136	5.193559784	5.195508356	0.141638616
		7	1.024906657	0.054101137	1.058342998	1.079007794	1.112444136	5.195508356	5.19092799	0.087537479
		8	1.024906657	0.033436342	1.045571452	1.058342998	1.079007794	5.196138679	5.195508356	0.054101137
		9	1.024906657	0.020664796	1.037678203	1.045571452	1.058342998	5.195680706	5.196138679	0.033436342
		10	1.037678203	0.012771546	1.045571452	1.050449749	1.058342998	5.196138679	5.196097499	0.020664796
		11	1.037678203	0.00789325	1.042556499	1.045571452	1.050449749	5.196040402	5.196138679	0.012771546
		12	1.042556499	0.004878297	1.045571452	1.047434796	1.050449749	5.196138679	5.19615213	0.00789325
		13	1.045571452	0.003014953	1.047434796	1.048586405	1.050449749	5.19615213	5.196142402	0.004878297
		14	1.045571452	0.001863343	1.046723062	1.047434796	1.048586405	5.196151253	5.19615213	0.003014953
		15	1.046723062	0.00115161	1.047434796	1.047874672	1.048586405	5.19615213	5.196150041	0.001863343
		16	1.046723062	0.000711734	1.047162938	1.047434796	1.047874672	5.196152416	5.19615213	0.00115161
		17	1.046723062	0.000439876	1.04699492	1.047162938	1.047434796	5.196152209	5.196152416	0.000711734
		18	1.04699492	0.000271858	1.047162938	1.047266778	1.047434796	5.196152416	5.196152398	0.000439876
		19	1.04699492	0.000168018	1.047098761	1.047162938	1.047266778	5.196152372	5.196152416	0.000271858
		20	1.047098761	0.000103841	1.047162938	1.047202601	1.047266778	5.196152416	5.196152423	0.000168018
		21	1.047162938	6.4177E-05	1.047202601	1.047227115	1.047266778	5.196152423	5.196152418	0.000103841
		22	1.047162938	3.96636E-05	1.047187451	1.047202601	1.047227115	5.196152422	5.196152423	**Converged**
		xm	***1.047195026***							
		f(xm)	***5.196152423***							

```r
f <- function(x) 4 * sin(x) * (1 + cos(x))

x.l <- 0
x.u <- pi/2
tol <- 10e-5

plot(f,type="l",from=-10,to=+10)

gsection <- function(ftm, x.l, x.u, tol) {
  Phi <- (sqrt(5)-1)/2 #golden ratio
  y.t <- NULL
  # evaluate the function at the bounds
  f.l <- ftm(x.l)
  f.u <- ftm(x.u)
```

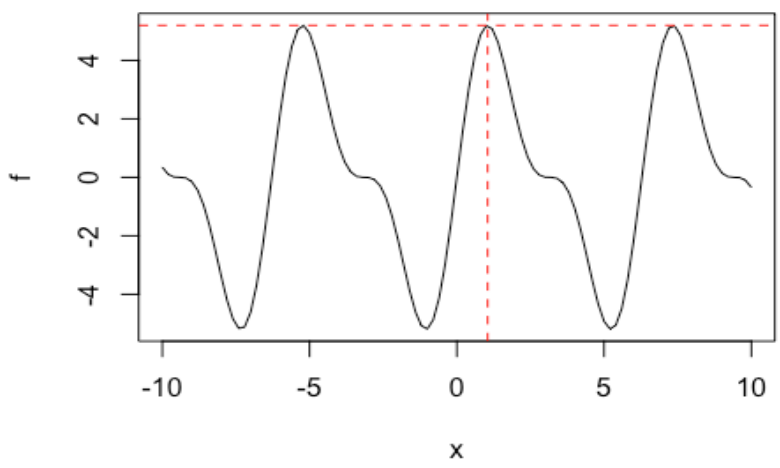
```r
  #compute the distance
  d <- Phi*(x.u-x.l)
  # compute intermediate points
  x.1 <- x.u - d
  x.2 <- x.l + d
  #evaluate the function at the intermediate points
  f.1 <- ftm(x.1)
  f.2 <- ftm(x.2)

  y.t <- (x.l+x.u)/2 #y.t collects the intermediates points
  i = 0
  while((x.u - x.l) > tol){
    i = i+1
    if (f.1 > f.2){ #max to the left
      x.u <- x.2; f.u <- f.2 #old x2 becomes new xu and fu=f2
      x.2 <- x.1; f.2 <- f.1 #old x1 becomes new x2 and f2=f1
      #compute the new distance
       d <- Phi*(x.u-x.l)
      #compute the new intermediate point and evaluate the function
      x.1 <- x.u - d; f.1 <- ftm(x.1)

      y.t <- c(y.t,(x.l+x.u)/2)
    }else{ #if f1<f2 the max is to the right
      x.l <- x.1; f.l <- f.1 #old x1 becomes new xl and fl=f1
      x.1 <- x.2; f.1 <- f.2 #old x2 becomes new x1 and f1=f2
      #compute the new distance
      d <- Phi*(x.u-x.l)
      #compute the intermediate point and evaluate the function
      x.2 <- x.l+d; f.2 <- ftm(x.2)

      y.t <- c(y.t,(x.l+x.u)/2)
    }
    x.m <- (x.l+x.u)/2 #x value of the maximum point
    f.m <- ftm(x.m) #evaluate f in the x of maximum point
  }
  abline(v=x.m, col="red", lty=2)
  abline(h=f.m, col="red", lty=2)
  return(out=list(y.t=y.t,x.m=x.m,f.m = f.m))
}


opt <- gsection(f, x.l, x.u, tol)
```




```
opt
```

```
## $y.t
##  [1] 0.7853982 1.0853936 0.8999862 1.0145743 1.0853936 1.0416248 1.0686754
##  [8] 1.0519572 1.0416248 1.0480106 1.0440640 1.0465031 1.0480106 1.0470789
## [15] 1.0476547 1.0472989 1.0470789 1.0472149 1.0471308 1.0471828 1.0472149
## [22] 1.0471950
##
## $x.m
## [1] 1.047195
##
## $f.m
## [1] 5.196152
```

The algorithm converged to an optimal x value of 1.047195 and a maximum function value of 5.196152.

EXERCISE 4

Use the Newton-Raphson algorithm to find the root of the following function:

f(x) = 2x^3 + 3x^2 + 5,

with starting values x0 = 0.5 and x0 = 0. On the basis of the the graph of the function, select alternative starting points and check the convergence to the true solution.

```
f4 <-  function(x) 2 * x^3 + 3 * x^2 + 5
f4prime <- function(x) 6*x^2 +6*x

#we use the function developed in exercise 1  for the other computation

#x0=0
#tab_0 <- newton_raphson(f4, f4prime, x0 = 0)
```

When x0=0 the first derivative is equal to 0, then the tangent line at x=0 is horizontal, meaning that it never intersects the x-axis to provide the next estimate xn+1. That represents a disadvantage using the Newton-Raphson method because it uses the derivatives, so, it depends on the behaviour of them. When the first or the second derviative is null, the method fails. The unique solution is changing the starting point.

```
#x0=0.5
tab_05 <- newton_raphson(f4, f4prime, x0 = 0.5)
tab_05
```

```
##    iter          x            fx         dfx     x_next
## 1     0  0.5000000  6.000000e+00   4.5000000         NA
## 2     0  0.5000000  6.000000e+00   4.5000000 -0.8333333
## 3     1 -0.8333333  5.925926e+00  -0.8333333  6.2777778
## 4     2  6.2777778  6.180521e+02 274.1296296  4.0231797
## 5     3  4.0231797  1.837961e+02 121.2549295  2.5073972
## 6     4  2.5073972  5.538934e+01  52.7666288  1.4576932
## 7     5  1.4576932  1.756942e+01  21.4953771  0.6403349
## 8     6  0.6403349  6.755198e+00   6.3021827 -0.4315474
## 9     7 -0.4315474  5.397963e+00  -1.4718854  3.2358322
## 10    8  3.2358322  1.041741e+02  82.2386554  1.9691030
## 11    9  1.9691030  3.190197e+01  35.0788184  1.0596662
## 12   10  1.0596662  1.074846e+01  13.0953519  0.2388819
## 13   11  0.2388819  5.198457e+00   1.7756785 -2.6887073
## 14   12 -2.6887073 -1.218668e+01  27.2426373 -2.2413688
## 15   13 -2.2413688 -2.448880e+00  16.6941926 -2.0946783
## 16   14 -2.0946783 -2.185128e-01  13.7579933 -2.0787957
## 17   15 -2.0787957 -2.405602e-03  13.4555750 -2.0786169
## 18   16 -2.0786169 -3.027637e-07  13.4521881 -2.0786169
```

```
# graph with x0=0.5
par(mfrow=c(1,1))
```

```r
curve(f4, from = -5, to = +5, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson", ylim=c(-50, +50))
abline(h = 0)

# extract x_n from the dataframe
xs <- tab_05$x
ys <- f4(xs)

#points
points(xs, ys, pch = 19, col = "red")

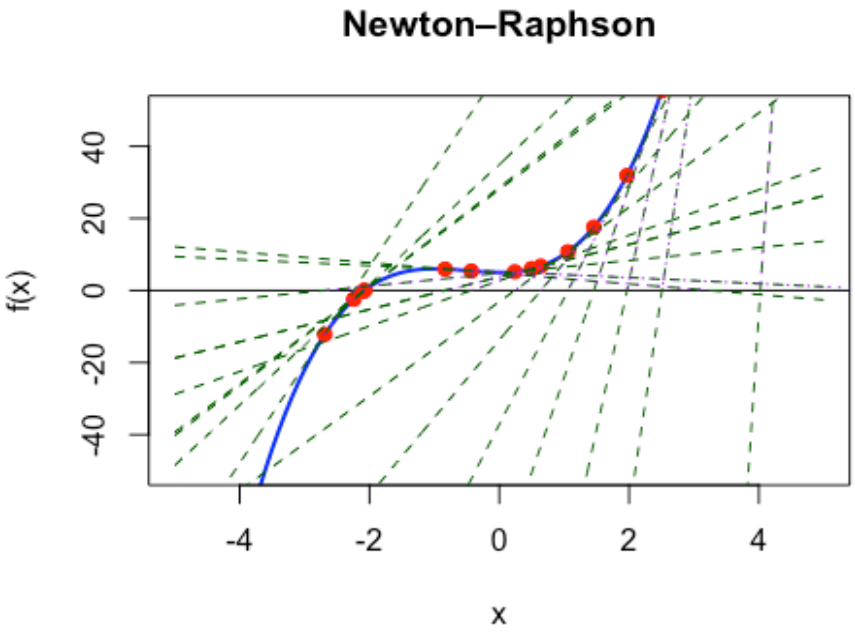
#tangents
for (i in 1:(nrow(tab_05)-1)) {

  x0 <- tab_05$x[i]
  y0 <- tab_05$fx[i]
  slope <- tab_05$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)

  segments(x0, y0, tab_05$x_next[i], 0, col = "purple", lty = 3)
}
```



The graph and the table illustrate divergence in the Newton-Raphson method. The red points (successive estimates xn) move progressively farther away from the true root. This behavior is due to the non-optimal initial choice of x0 = 0.5 which is close to a local minimum (x=0). When an iteration lands near a point where the derivative is close to zero, the subsequent step length (-f(x)/f'(x)) becomes excessively large, causing the estimate to "jump" far away and fail to converge. We suggest point that are far from the critical value x=0, as x0=-3 or x0=-2

```
#x0=-3
tab_3 <- newton_raphson(f4, f4prime, x0 = -3)
tab_3 #rapid converge to the root

##   iter         x            fx      dfx    x_next
## 1    0 -3.000000 -2.200000e+01 36.00000        NA
## 2    0 -3.000000 -2.200000e+01 36.00000 -2.388889
## 3    1 -2.388889 -5.145405e+00 19.90741 -2.130422
## 4    2 -2.130422 -7.225907e-01 14.44966 -2.080415
## 5    3 -2.080415 -2.421353e-02 13.48626 -2.078619
## 6    4 -2.078619 -3.055559e-05 13.45223 -2.078617

par(mfrow=c(1,1))
curve(f4, from = -5, to = +5, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson", ylim=c(-50, +50))
abline(h = 0)

# extract x_n from the dataframe
xs <- tab_3$x
ys <- f4(xs)

# points
points(xs, ys, pch = 19, col = "red")

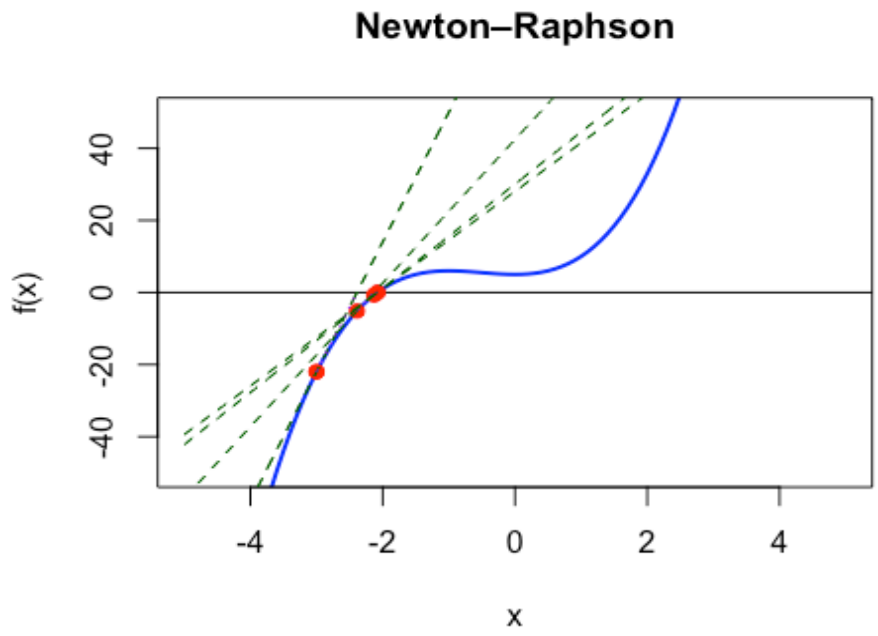
# tangents
for (i in 1:(nrow(tab_3)-1)) {

  x0 <- tab_3$x[i]
  y0 <- tab_3$fx[i]
  slope <- tab_3$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)

  segments(x0, y0, tab_3$x_next[i], 0, col = "purple", lty = 3)
}
```



This graph shows fast convergence for x0 = -3. The sequence of red points quickly moves closer to the root.

```
#x0=-2
tab_2 <- newton_raphson(f4, f4prime, x0 = -2)
tab_2 #very fast convergence to the root.
```

```
##   iter         x            fx      dfx    x_next
## 1    0 -2.000000  1.000000e+00 12.00000        NA
## 2    0 -2.000000  1.000000e+00 12.00000 -2.083333
## 3    1 -2.083333 -6.365741e-02 13.54167 -2.078632
## 4    2 -2.078632 -2.097236e-04 13.45248 -2.078617
## 5    3 -2.078617 -2.302077e-09 13.45219 -2.078617
```

```
par(mfrow=c(1,1))
curve(f4, from = -5, to = +5, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Newton-Raphson", ylim=c(-50, +50))
abline(h = 0)

# extract x_n from the dataframe
xs <- tab_2$x
ys <- f4(xs)

# points
points(xs, ys, pch = 19, col = "red")

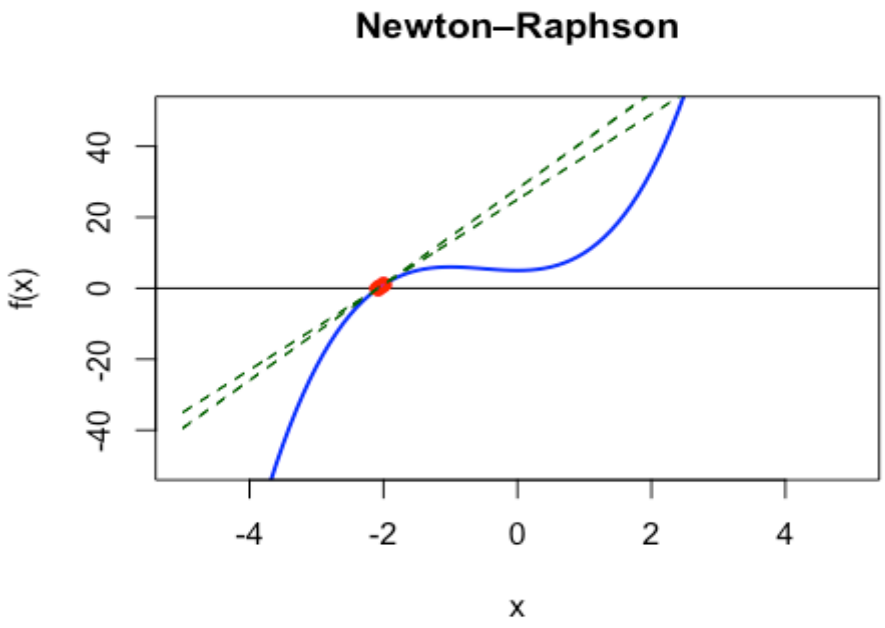
# tangents
for (i in 1:(nrow(tab_2)-1)) {
```

```
  x0 <- tab_2$x[i]
  y0 <- tab_2$fx[i]
  slope <- tab_2$dfx[i]

  tangent <- function(x) y0 + slope*(x - x0)

  curve(tangent, add = TRUE, col = "darkgreen", lty = 2, n = 1000)


  segments(x0, y0, tab_2$x_next[i], 0, col = "purple", lty = 3)
}
```



This graph illustrates very rapid convergence starting from x0=-2, highlighting that starting close to the root significantly improves the speed of the Newton-Raphson method.

EXERCISE 5

Consider maximizing the likelihood of a Cauchy(θ,1), with θ the unknown location parameter

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} \frac{1}{\pi[1 + (x_i - \theta)^2]}$$

Note that the Cauchy distribution is often used in statistics as the canonical example of a "patho-logical" distribution since both its expected value and its variance are undefined. The median and the mode of this distribution are given by the location parameter θ.

a) Write the log-likelihood of the Cauchy distribution as function of parameter θ;

```
L_theta <- function(theta, x) {
  -length(x) * log(pi) - sum(log(1 + (x - theta)^2))
}
```

b) Simulate a sample of Nobs = 400 observations from the Cauchy distribution using the inversevtransformation method where the cdf of the Cauchy is

FX(x) = 1/pi arctan(x-theta)+1/2

for a value of the location parameter θ= 0;

```
n <- 400
theta1 <- 0
set.seed(123)
u <- runif(n)
x <- tan(pi*(u-0.5)+theta1) #inverse with theta = 0
```

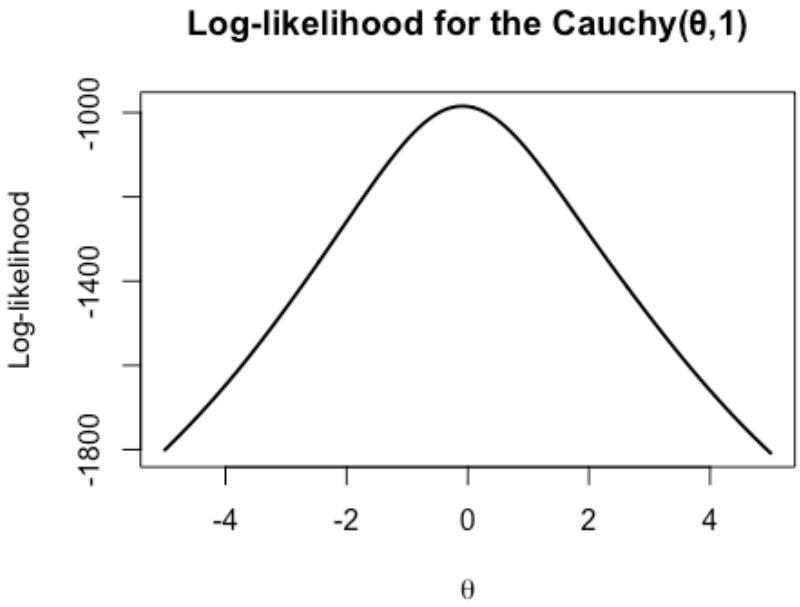
c) Given the simulated data, plot the graph of the log-likelihood as a function of parameter θ;

```
#we need to create an interval to generate theta because we need to compute the lo
g-likelihood on each value of theta

theta <- seq(-5, 5, length.out = 500)

#now we calculate the log likelihood on the values of theta
LL_values <- sapply(theta, L_theta, x = x)

#plot
plot(theta, LL_values, type = "l", lwd = 2,
     xlab = expression(theta),
     ylab = "Log-likelihood",
     main = "Log-likelihood for the Cauchy(θ,1)")
```




```
#looking at the graph we can see that the maximum point is reached at a value very
 close to zero (negative)
```

d) Maximize the log-likelihood using the golden section search algorithm and the Newton-Raphson method (try different starting values to check the convergence) and compare the results.

```
#GOLDEN SECTION ALGORITHM
f1 <- function(theta) L_theta(theta, x)

golden_section_algorithm <- function(f1, x.l, x.u, tol) {
  Phi <- (sqrt(5)-1)/2
  y.t <- NULL

  f.l <- f1(x.l)
  f.u <- f1(x.u)

  d <- Phi*(x.u-x.l)
  x.1 <- x.u - d
  x.2 <- x.l + d

  f.1 <- f1(x.1)
  f.2 <- f1(x.2)

  y.t <- (x.l+x.u)/2

  i = 0
  while((x.u - x.l) > tol){
```

```
    i = i+1
    if (f.1 > f.2){
      x.u <- x.2; f.u <- f.2
      x.2 <- x.1; f.2 <- f.1

      d <- Phi*(x.u-x.l)

      x.1 <- x.u - d; f.1 <- f1(x.1)

      y.t <- c(y.t,(x.l+x.u)/2)
    } else {
      x.l <- x.1; f.l <- f.1
      x.1 <- x.2; f.1 <- f.2

      d <- Phi*(x.u-x.l)

      x.2 <- x.l+d; f.2 <- f1(x.2)

      y.t <- c(y.t,(x.l+x.u)/2)
    }

    x.m <- (x.l+x.u)/2
    f.m <- f1(x.m)
  }

  return(out=list(y.t=y.t, x.m=x.m, f.m=f.m))
}

#we choose an interval and the tolerance level
x.l <- min(x) - 5
x.u <- max(x) + 5
tol <- 1e-5

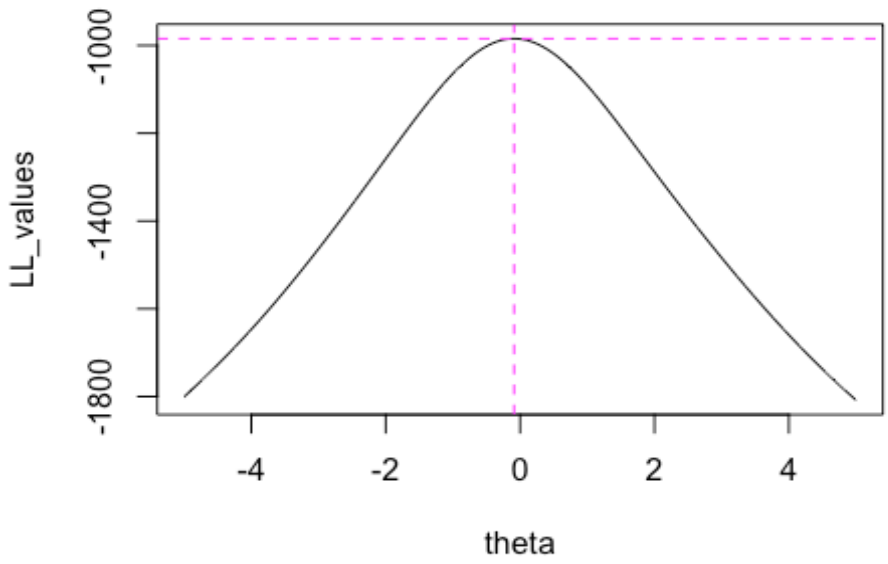
theta_golden <- golden_section_algorithm(f1, x.l, x.u, tol); theta_golden

## $y.t
##  [1]   12.53496876 -188.76681447  -64.35547044   12.53496876  -34.98593607
##  [6]   -5.61640171   12.53496876    1.31680487   -5.61640171   -1.33144439
## [11]    1.31680487   -0.31990319    0.69163802    0.06647117   -0.31990319
## [16]   -0.08111070   -0.22869257   -0.13748196   -0.08111070   -0.11595005
## [21]   -0.09441815   -0.08111070   -0.08933516   -0.09441815   -0.09127669
## [26]   -0.08933516   -0.09053509   -0.08979349   -0.09025182   -0.08996856
## [31]   -0.08979349   -0.08990169   -0.08983482   -0.08987614   -0.08985060
## [36]   -0.08986639   -0.08985663   -0.08986266   -0.08986639   -0.08986409
##
```

```
## $x.m
## [1] -0.08986409
##
## $f.m
## [1] -984.5446

theta_golden <- golden_section_algorithm(f1, x.l, x.u, tol)

#plot
plot(theta, LL_values, type = "l")
abline(v = theta_golden$x.m, col="magenta", lty=2)
abline(h = theta_golden$f.m, col="magenta", lty=2)
```



The value -0.08986409 corresponds to the maximum point which can be also seen in the graph and with the intersection with the two magenta lines. We know that the results provided by the golden section algorithm are robust because it does not use the derivatives. The process is computed in a finite interval and it can not explode during computations.

We could also use the R function “optimize” but we prefer to compute this algorithm.

```
#NEWTON-RAPHSON METHOD

#first derivative
loglik_prime <- function(theta, x) {
  sum( 2*(x - theta) / (1 + (x - theta)^2) )
}
```

```r
#second derivative
loglik_second <- function(theta, x) {
  sum( 2 * ((x - theta)^2 - 1) / (1 + (x - theta)^2)^2 )
}

#NR method
newton_raphson_method <- function(theta, x, tol = 1e-8, maxiter = 100) {
  for (i in 1:maxiter) {
    theta_new <- theta - loglik_prime(theta, x) / loglik_second(theta, x)
    if (abs(theta_new - theta) < tol) break
    theta <- theta_new
  }
  return(theta)
}

theta_newton <- newton_raphson_method(theta = median(x), x = x); theta_newton
```

```
## [1] -0.08986378
```

```r
#the results obtained maximizing by using the golden section algorithm or the
#Newton-Raphson method (with starting point equal to the median) are the same,
#they change starting from the fifth decimal

#now we try different starting values

starts <- c(median(x), mean(x), -1, 1, 5, -5)

#table to see a clear representation
table <- data.frame(
  starting_value = starts,
  theta_hat = sapply(starts, function(s) newton_raphson_method(theta = s, x = x)),
  logLikelihood = sapply(starts, function(s) L_theta(newton_raphson_method(theta =
 s, x = x), x))
); table
```

```
##   starting_value     theta_hat logLikelihood
## 1    -0.06979139 -8.986378e-02     -984.5446
## 2     0.29239602 -8.986378e-02     -984.5446
## 3    -1.00000000 -8.986378e-02     -984.5446
## 4     1.00000000 -8.986378e-02     -984.5446
## 5     5.00000000  4.493942e+30   -56922.1185
## 6    -5.00000000 -4.989981e+29   -55163.8118
```

Newton–Raphson converges to the correct maximum only when it starts from values close to it: with distant starting points (such as 5 or −5) it diverges and produces unrealistic estimates, because the Cauchy log-likelihood is non-concave and makes the method unstable. Moreover, the Newton-Raphson method uses derviatives, it means that if

the second derivative is close to zero, the process can diverge. The golden section, instead, is robust and always identifies the maximum.